Problem Statement

As Large Language Models (LLMs) become increasingly sophisticated, distinguishing between text generated by different AI models has become a critical challenge in AI safety, security, and content moderation.

Your task is to build a machine learning model that can identify the author/source of a given piece of text. You will be working with a massive, perfectly balanced dataset containing text generated by 12 different LLMs (including ChatGPT, GPT-4, Llama, and Mistral). Given a raw text string, your model must accurately predict which of the 12 models generated it.

Dataset

You are provided with a 12-class text classification dataset derived from the Hugging Face RAID dataset. It is perfectly balanced, meaning no model is over-represented.

Structure:

- `train/train.parquet`: 960,000 rows (Use this to train your models)
- `val/val.parquet`: 99,600 rows (Use this to tune hyperparameters and check for overfitting)
- `test/test.parquet`: 99,600 rows (Use this **only once** at the very end to report your final accuracy for each model)

Columns:

- `model`: The target label (e.g., `chatgpt`, `gpt4`, `llama-chat`).
- `generation`: The raw text produced by the model.

Steps to build a model

Machine learning models are mathematical engines; they cannot read raw English. Before building a complex model, you must figure out your data pipeline.

Step 1: Data Setup

1. **Load the Data:** Use Pandas or the Hugging Face `datasets` library to load the Parquet files.
2. **Subsample for Debugging (Crucial Step):** 960,000 rows will take hours to process. Create a "mini-dataset" of just 1,000 rows per class. Use this mini-dataset to write,

debug, and test your code so it runs in seconds. **Only use the full dataset once you know your code works entirely without errors.**

3. **Explore the Data:** Print out a few examples from different classes. Can you notice any obvious patterns between `gpt4` and `gpt2` just by reading them?

Step 2: Vectorization / Embedding

1. **Loading the Data:** Read the Parquet files into a workable format (like a Pandas DataFrame).
2. **Tokenization:** Break the raw text into smaller pieces (words, subwords, or characters).
3. **Vectorization / Embedding:** Convert those tokens into numbers that capture either the frequency or the semantic meaning of the text. Here are the two primary approaches for Vectorization / Embedding.

Traditional Machine Learning (TF-IDF)

It converts text into a matrix of word frequencies, penalising words that appear too frequently across all texts (such as "the" or "and").

How the input works here:

- **Input Text:** `"The quick brown fox"`
- **Vectorized Input:** `[0.0, 0.12, 0.0, 0.85, ...]` (A sparse array where each index represents a specific word in your vocabulary).

Deep Learning based pre-trained Embeddings

Word Embeddings (Word2Vec/GloVe): Use a pre-trained dictionary to convert those integer IDs into rich, multi-dimensional vectors that capture semantic meaning.

Step 3: Design, train, and evaluate both traditional machine learning and deep learning models.

Step 4: Compare model architectures based on performance, training time, and computational cost.